TRABAJO PRACTICO INTEGRADOR

Calidad de Software

CapsaAI / SaverAI

Módulo de persistencia de gastos e integración con PostgreSQL

Nivel declarado	Fecha
1 - inicial	26 de octubre de 2026

Integrantes

Geronimo Crostelli

Baltasar Lopez Villagra

Facundo Ortea

Modalidad declarada: trabajo conjunto. La autoría individual retrospectiva no está discriminada; las evidencias personales de defensa se registrarán por cada integrante.

Pruebas satisfactorias	Cobertura existente	Cobertura nueva
94 unitarias + 27 integración	137/228 = 60,09 %	45/50 = 90 %

Estado: implementación técnica, reportes y demostración local preparados. Se declaran pendientes la aceptación académica del alcance/localidad, la trazabilidad individual. No se afirma cumplimiento de toda la tesis.

Repositorio: Geronimo2001/SaverAI

Base de la entrega: main 05c0206, posterior al PR #6. El PDF conserva las cifras de evidencias identificadas; no mezcla mediciones locales con CI.

1. Alcance y objetivos

Se evalúa el backend Node.js/TypeScript de registro de gastos: solicitud firmada, validación, referencias, transacción en PostgreSQL, consulta y deshacer. Se incluyen todos los archivos TypeScript de server/ en la cobertura unitaria.

La interfaz Expo/React Native, el login SQLite y la conversación con WhatsApp son contexto de la tesis y quedan fuera de la verificación funcional de este módulo. No se afirma que la app completa tenga estos porcentajes o haya sido probada de extremo a extremo.

Perspectiva	Objetivo medible	Evidencia
Usuario	No guardar entradas inválidas del conjunto ensayado; reintentos sin duplicar.	Validación, API, filas SQL y regresiones.
Equipo	Existente >=50 %, nuevo >=80 %; regresión por defecto.	Gate y pruebas ejecutadas.
Organización	Cambios pasan por PR y check quality obligatorio.	Protección de main y PR #6.
Continuidad	No confirmar una escritura fallida ni dejarla parcial.	Rollback real y dobles de fallo.

Arquitectura y herramientas

HTTP firmado > validador > repositorio > pg > PostgreSQL 15. Las lecturas y el borrado retornan los datos del usuario indicado. La base de pruebas capsaai_quality_test tiene nombre y marcador verificados antes de migrar o crear fixtures.

Finalidad	Herramientas
Desarrollo y compilación	Node 24, TypeScript, pg; Expo en la interfaz contextual.
Pruebas y análisis	Vitest + V8, ESLint + SonarJS, jscpd.
Versionado y automatización	Git, GitHub Actions, protección de main.
Carga y entrega	k6 2.3.0; scripts de demo y generación de PDF.

QA, QC y contingencia

QA: criterios previos, contrato, gestión de riesgos, entorno aislado y revisión por PR. **QC:** ejecutar casos, analizar métricas, registrar defectos y verificar correcciones.

Ante caída de PostgreSQL no se debe anunciar un gasto guardado. Se comprobó rollback y rechazo de fallos con dobles; la API conserva 500 genérico. Reintentos operativos, timeouts productivos y una respuesta específica de indisponibilidad son mejoras pendientes. No se ofrece almacenamiento offline inexistente.

2. Modelo de calidad y riesgos

ISO/IEC 25010	Aplicación y evidencia	Estado
Adecuación funcional	Validación, referencias, alta, consulta y borrado; CP-001 a CP-017.	Verificado en el módulo.
Eficiencia de desempeño	Load/stress en CI y soak corto local; errores y percentiles.	Acotado a perfiles publicados.
Compatibilidad	Contrato HTTP/pg, SQL y restricciones reales; CP-009 a CP-020.	Verificado con PostgreSQL.
Usabilidad	Mensajes de error de API y validación; interfaz no ensayada.	Parcial; sin estudio de usuarios.
Fiabilidad	Atomicidad, rollback, idempotencia y concurrencia; CP-010/013/019.	Verificado en casos definidos.
Seguridad	HMAC, SQL parametrizado y conflicto ajeno. Lecturas sin autenticación.	Parcial; riesgo de acceso abierto.
Mantenibilidad	Tipos, suite, reportes, regresiones, duplicación y gate.	Verificado por herramientas.
Portabilidad	Migración repetible, macOS local y CI Linux; demo de copia local.	Parcial; sin despliegue cloud.

Se emplean las ocho características pedidas por la guía. La matriz no es una certificación ISO ni sustituye evidencias de características fuera del alcance.

Top 3: adecuación funcional, porque el gasto debe ser correcto; fiabilidad, para evitar pérdidas/duplicados; seguridad, por tratar datos financieros.

Riesgos priorizados

Riesgo	P x I	Tratamiento
Lectura ajena por identificación no autenticada	3 x 3 = 9	Limitación abierta; filtrar por userId no autentica.
Alterar la base original con pruebas	2 x 3 = 6	Nombre local explícito, marcador, datos propios y limpieza acotada.
Duplicado concurrente / alta parcial	2 x 3 = 6	Advisory locks, transacción, restricciones y regresiones.
Monto aceptado se convierte en cero	3 x 2 = 6	DEF-001: validar también el monto redondeado.
Alcance/localidad no aceptados	2 x 3 = 6	Confirmar con cátedra; no afirmar aceptación pendiente.

P e I son estimaciones iniciales de 1 a 3. No son frecuencias medidas. Las restricciones y locks no resuelven autenticación de lecturas ni prueban seguridad completa.

3. Plan y trazabilidad

Entrada: alcance, aceptación previa, herramientas instalables, tipos correctos, fixtures sintéticos, base protegida y baseline fijado. Salida técnica: casos críticos satisfactorios, gate, reportes y regresiones. El cumplimiento académico completo conserva los pendientes declarados.

Requisito	Casos / nivel	Vínculo
REQ-01 Monto	CP-001/002/003/004; unitaria y API.	DEF-001; entero positivo hasta 100.000.000.
REQ-02 Campos	CP-005/006; unitaria.	IDs/campos obligatorios; comercio 2-80 caracteres.
REQ-03 Fecha	CP-007/008; unitaria.	Fecha real y no futura; reloj fijo.
REQ-04 Referencias	CP-009; dobles y PostgreSQL.	Usuario/categoría existentes; medio propio activo.
REQ-05 Idempotencia	CP-010/011; integración.	DEF-002; reintentos y origen compartido.
REQ-06 Atomicidad	CP-012/013; dobles e integración.	Commit/rollback y liberación de cliente.
REQ-07 Consulta	CP-014/015; API/SQL.	DEF-004; filtro y formato de período.
REQ-08 Deshacer	CP-016/017; API/SQL.	Borrado propio y cascada; rechazo de ajeno.
REQ-09 Firma	CP-018; unitaria y API.	HMAC válida/ausente/alterada.
REQ-10 Caída	CP-019; dobles y error SQL.	No confirmar escrituras parciales.
REQ-11 Entorno	CP-020; integración.	Migración sobre destino aislado, repetible.
REQ-12 Despliegue	CP-021; smoke y demo local.	Copia nueva compilada, HTTP y base reales.
REQ-13 Duplicado ajeno	CP-022; API e integración.	DEF-003; sin datos del gasto ajeno.

Los IDs son casos documentados, no el número de assertions: las tablas parametrizadas generan varios tests por caso. La suite actual tiene 94 unitarias y 27 integraciones; smoke y carga se informan aparte.

Plan completo, criterios iniciales y riesgos

Contrato HTTP/PostgreSQL y tabla de decisión histórica

4. Diseño de pruebas

Técnica	Entradas representativas	Esperado
Equivalencias de monto	No numérico; <=0; positivo que redondea a cero; válido; >máximo.	Rechazar inválidos antes de persistir.
Límites de monto	-1, 0, 0.1, 0.49, 0.5, 1, 100.000.000, 100.000.001.	0.1/0.49 rechazan; 0.5 sigue redondeando a 1.
Límites de comercio	1, 2, 80 y 81 caracteres.	Aceptar solo 2 a 80 tras normalizar.
Fechas	Hoy, ayer, mañana, bisiesto, mes/día inválidos.	Aceptar reales no futuras con reloj fijo.
Período de consulta	2026-13, 2026-00, 0000-01, texto, vacío.	400 antes de consultar SQL.

Decisiones de referencias e idempotencia

Condición	Decisión comprobada
Mensaje nuevo + referencias correctas	Crear una vez.
Usuario/categoría inexistente o medio ajeno/inactivo	422 y sin escrituras.
Mensaje ya usado por el mismo usuario	200 duplicado con mismo gasto.
Mensaje ya usado por otro usuario	422 message_conflict, sin devolver gasto ajeno.
Dos solicitudes con origen común	Un creado y un duplicado; una sola fila.

Arrange - Act - Assert

CP-003 prepara un payload válido y cambia amount a 0.1 (Arrange); llama al validador con fecha de referencia fija (Act); exige valid:false y error invalid_amount (Assert). Las expectativas se mantienen entre rojo y verde. No se cambia la política monetaria general para hacer pasar el caso.

Siete principios aplicados

1. Un verde muestra que esos casos pasan; no demuestra ausencia de defectos.
2. No se prueba todo: se priorizan montos, referencias, transacciones y separación.
3. Se fijan aceptación y casos antes de la corrección.
4. Los defectos se concentraron en validación, repositorio y límites con SQL.
5. Las regresiones evolucionan con errores nuevos y datos límite.
6. En gastos financieros pesan integridad, privacidad y trazabilidad.
7. Ausencia de errores no prueba utilidad: falta validar la experiencia de la app.

5. Defectos y mejora comprobada

ID / casos	Esperado frente a lo obtenido	Corrección y estado
DEF-001 / CP-003	Monto 0.1 debía rechazarse; se aceptaba y redondeaba a cero, incompatible con CHECK amount>0.	Validar monto redondeado >0. Corregido; regresión local y CI.
DEF-002 / CP-011	Dos altas con origen común debían crear un gasto; ambas podían responder 201.	Locks de transacción ordenados; no ocultar conflictos. Corregido.
DEF-003 / CP-022	Mensaje ajeno debía rechazarse sin datos; devolvía gasto del otro usuario.	Comparar pertenencia y responder conflicto genérico. Corregido.
DEF-004 / CP-015	Período inválido debía devolver 400; generaba 500 o se ignoraba.	Validar formato, mes y año antes de SQL. Corregido.

Contraste real local

En la etapa 3 la misma suite de integración pasó de **7 fallos de 26 casos** (26,92 %) a **0 de 26**. Los fallos eran un caso de DEF-002, uno de DEF-003 y cinco entradas de DEF-004. El hash del test coincide entre rojo y verde: no son siete defectos distintos.

Esta es una mejora medida de comportamiento, no una comparación de rendimiento ni una afirmación de ausencia total de errores. La suite posterior se priorizó y sus conteos actuales son distintos.

Contraste real en CI

Se reprodujo históricamente DEF-001 en una rama independiente, retirando solo el control del monto redondeado. Fallaron 4/94 unitarias: tres entradas del validador y una comprobación HTTP previa a persistir. Las 27 integraciones pasaron. Luego se restauró la corrección y pasaron las 121 pruebas, sin cambiar tests, workflow ni umbrales.

CI roja 40265714889 | CI verde 40265838359

En rojo no se generó LCOV; el gate también rechazó el reporte ausente. No hay porcentajes rojos sustituidos por una ejecución anterior. La reproducción no fue un descubrimiento nuevo ni se fusionó el defecto a main.

Registro completo versionado en docs/calidad/03-defectos.md: pasos, esperado/obtenido, severidad, corrección y evidencia. Los defectos están registrados retrospectivamente en GitHub Issues #7 a #10 y cerrados con evidencia de corrección previa; la fecha del issue no mide su resolución. Falta registrar aportes personales.

DEF-001 / issue #7 | DEF-002 / issue #8 | DEF-003 / issue #9 | DEF-004 / issue #10

Registro detallado de los cuatro defectos

Identidad de las pruebas y commits en CI roja/verde

6. Métricas e interpretación

Métrica	Resultado vigente	Criterio / lectura
Código existente	137 / 228 = 60,09 %	Mínimo 50 %; satisface el gate.
Código nuevo	45 / 50 = 90 %	Mínimo 80 %; satisface el gate.
Líneas globales del backend	182 / 278 = 65,46 % (V8)	No es cobertura de toda la app.
Sentencias	194/298 = 65,10 %	Cobertura unitaria de sentencias.
Decisiones / ramas	114/168 = 67,85 %	Ramas instrumentadas por V8; aproximación de decisiones, no MC/DC.
Duplicación	0 % sobre 915 líneas	jscpd: 5 líneas/50 tokens, modo mild; máximo 5 %.
Análisis estático	0 errores / 0 advertencias	ESLint/TS/SonarJS; no equivale a seguridad certificada.
Duración del pipeline (proceso)	83,08 s	Fin - inicio del runner CI de la ejecución 40328308100; sin cola.
Regresión por defecto (proyecto)	4 de 4 = 100 %	Cada defecto corregido conserva caso ejecutado; no mide autoría.

Qué significa código nuevo

La referencia fija es cf5cc7d7184c79f32acfd22ecaa8b9dca8ca96c1. El gate compara líneas ejecutables actuales con esa referencia: sin cambio son existentes; agregadas/modificadas son nuevas. Los tests quedan fuera del denominador. Mover código a un archivo nuevo cuenta como nuevo según la regla publicada.

Ejemplo: que haya 90 % nuevo significa que las unitarias ejecutaron 45 de 50 líneas nuevas instrumentadas del backend. No significa que 90 % de los tests sean nuevos ni que el TP esté 90 % terminado.

Evolución y ajuste deliberado

Al inicio no había medición: no se informa 0 % inventado. La primera suite completa dio 87,28 % existente y 94 % nuevo, con 147 pruebas. Por decisión del equipo se conservaron casos prioritarios y se redujeron casos secundarios/duplicados: ahora 60,09 % y 90 %, con 121 pruebas. No cambiaron baseline, archivos medidos ni umbrales.

La reducción no se presenta como mejora de cobertura. La mejora funcional comprobada es la corrección de defectos con pruebas estables. Los reportes de etapas distintas se conservan separados para evitar mezclar numeradores y denominadores.

7. Pipeline y quality gate

Flujo: rama de trabajo > push > checks; pull request > checks de la combinación con main > aprobación y merge manual. El workflow se activa por push a main/codex/** y por pull request. Por eso un PR puede mostrar dos ejecuciones quality: push y PR.

Etapa	Qué comprueba
Tipos y compilación	App/backend/pruebas con TypeScript; JavaScript real en dist/server.
Análisis y duplicación	Backend completo con ESLint/SonarJS y jscpd.
Unitarias e integración	94 unitarias con V8 y 27 integraciones HTTP/SQL.
Gate de cobertura y calidad	Existente >=50 %, nuevo >=80 %, duplicación <=5 %, análisis y suites correctos.
Carga y stress	Perfiles load/stress, umbrales, recorridos completos, recuperación y limpieza.
Reportes automáticos	JSON, cobertura, logs, resumen de Actions y artefacto incluso al fallar.

Bloqueo de merges verificado

Main está protegido: requiere PR, check quality de GitHub Actions y una aprobación; se aplica también a administradores. La regla observada no exige que la rama esté actualizada. El equipo fusionó el PR #6 después de los checks. No se crea ni fusiona un PR desde esta entrega.

El gate devuelve error al incumplir sus reglas; la protección obliga a respetar el check antes del merge. Carga/stress fallidos también rechazan el job aunque la parte de cobertura esté aprobada. El verde habilita la acción manual; no despliega ni fusiona automáticamente.

La política bloquea fallos de tests y hallazgos del analizador. SonarJS en ESLint no es SonarQube ni usa sus severidades; no demuestra detectar todos los bugs críticos posibles. El acceso de lectura sin autenticación sigue siendo una limitación conocida.

Comandos

npm run quality:check - pipeline completo, load y stress.

npm run test:all - unitarias, integración, smoke y los tres perfiles.

npm run test:smoke - comprobación local mínima.

npm run demo:quality - copia nueva desplegada localmente, smoke y cargas.

CI con carga/stress y reportes 40328308100 | PR #6 fusionado por el equipo

8. Herramienta nueva: cinco pasos

Paso	Decisión y evidencia
1. Requisitos	HTTP/HMAC, carga constante y creciente, checks, percentiles, umbrales, JSON, CI, macOS/Linux y ejecución sin cuenta cloud.
2. Mercado	Comparación documental oficial de k6, Artillery y Locust. No se afirma haber ejecutado las tres herramientas.
3. PoC	k6 2.3.0: 1 VU, 1 ciclo, 4 solicitudes/checks correctos. Prueba separada de umbral imposible: k6 salió 99, como se esperaba.
4. Matriz	Escala 1-5 y pesos explícitos. k6 4,60; Artillery 4,50; Locust 3,75. Juicio técnico, no benchmark.
5. Adopción incremental	PoC > perfiles locales > load/stress en CI con reportes; soak manual. Ampliación remota/duración mayor pendiente.

Criterio / peso	k6	Artillery	Locust
Afinidad de scripts / 30 %	4	5	2
Umbrales/CI / 25 %	5	4	4
Perfiles/métricas / 20 %	5	5	5
Firma/checks / 15 %	5	4	5
Instalación local/CI / 10 %	4	4	4
Ponderado / 5	4,60	4,50	3,75

k6 se eligió por sus umbrales y escenarios explícitos. El equipo conoce JS/TS, pero su runtime es propio y no es Node. La versión se descarga en .tools/ ignorado, con SHA-256 de distribución y verificación de cache; no se instala globalmente ni envía datos a Grafana Cloud.

El ensayo del umbral imposible verifica el comportamiento de la herramienta. No es un defecto de la aplicación ni reemplaza el CI rojo de DEF-001. Los perfiles reales conservan los límites definidos antes de ejecutarlos.

Evaluación completa, fuentes oficiales y PoC

Plan previo de carga, perfiles y límites

9. Rendimiento: plan y resultados

Cada VU usa cuenta propia y un mensaje único por iteración: alta firmada > consulta > borrado firmado > consulta vacía > pausa de 250 ms. Modelo cerrado: espera respuestas; la tasa observada no es una llegada impuesta.

Perfil	Plan	Criterios previos
load	2 VUs, 20 s.	0 fallos HTTP; 100 % de checks; ciclos completos.
stress	1 a 5, 10 y 20 VUs; mantener y bajar a 0; 50 s.	Mismos criterios, p95 <500 ms y p99 <1000 ms.
soak	2 VUs, 120 s.	Mismos límites; limpieza y recuperación verificadas.

Entorno / perfil	Ciclos	HTTP	p95 ms	p99 ms	Errores
Local / load	156	624	4,52	5,31	0 %
Local / stress	2083	8332	8,91	13,88	0 %
CI Linux / load	149	596	24,78	92,76	0 %
CI Linux / stress	1849	7396	49,57	138,48	0 %
Local / soak	930	3720	4,32	5,00	0 %

En todos esos perfiles los ciclos iniciados terminaron, los checks fueron correctos, no quedaron gastos/mensajes y la limpieza final pasó. Los números de la nueva demo se conservan por separado; no se sustituyen mediciones previas.

Entorno e interpretación

Local: Apple M4, 10 CPUs reportadas, macOS arm64, Node 24.15.0 y PostgreSQL 15.17. CI: Linux x64, 4 CPUs, Node 24.21.0 y PostgreSQL 15.19. k6 2.3.0 en ambos. Generador, backend y base comparten equipo.

El p95 de stress en CI fue 49,57 ms: el percentil describe la muestra, no una promesa para todo usuario futuro. La diferencia con local no mide regresión entre versiones porque cambia el entorno. Se cumplieron los umbrales iniciales elegidos para este ensayo.

No se alcanzó ni se certificó saturación. Dos minutos no descartan fugas durante horas. No se midieron CPU, memoria o recursos SQL; la carga usa cuentas separadas y no mide contención del mismo mensaje. La concurrencia de duplicados tiene sus propios casos funcionales.

Resultados, entornos y artefactos de la etapa de carga

10. Demostración y preparación

Se ejecutó npm run demo:quality. El comando compiló y creó un directorio nuevo con una copia del backend, package.json y un enlace a las dependencias existentes. Registró los hashes del artefacto y probó ese punto de entrada mediante instancias temporales. No fue una instalación independiente de dependencias ni una publicación cloud.

Control de la demo	Resultado
Nueva copia local	Creada y archivos identificados por SHA-256.
Smoke	10 comprobaciones satisfactorias: arranque, salud, catálogo, alta, consulta, borrado y limpieza.
Carga/stress/soak	Los tres perfiles completos y limpios sobre el artefacto copiado.
Retiro del artefacto	releaseRemoved=true; copia temporal retirada.
Duración total	191,24 s, incluyendo coordinación y cierre.

Perfil en nueva demo	Ciclos	HTTP	p95 ms	p99 ms
load	156	624	4,79	5,34
stress	2120	8480	4,73	5,74
soak	930	3720	4,39	4,89

Guion breve

1. Presentar nivel 1 y alcance; distinguir tesis completa de módulo probado.
2. Seguir REQ-01 > CP-003 > DEF-001 y explicar rojo/corrección/verde.
3. Mostrar PR/checks, gate y cobertura existente/nueva; identificar límites.
4. Ejecutar demo:quality, mostrar manifiesto, smoke, cargas y limpieza.
5. Interpretar VUs/p95, reconocer pendientes y explicar decisiones del equipo.

Aporte individual

Los tres integrantes declararon trabajo conjunto. No se asignaron autores ficticios. Para cerrar la trazabilidad individual, cada uno debe registrar una ejecución/revisión real con nombre, fecha, comando, commit y evidencia, y poder explicar el conjunto. La tabla está preparada, sin completar acciones futuras.

La demostración local está ejecutada; la exposición del grupo ante la cátedra todavía no ocurrió. La aceptación de local como despliegue y PostgreSQL como integración externa sigue sin confirmar.

Guion completo y operación segura de la demostración

Tareas, declaración conjunta y práctica individual pendiente

11. Cumplimiento y conclusiones

Meta de la guía	Estado al cierre técnico
Plan versionado, riesgos, matriz ISO y trazabilidad	Documentados con casos y evidencias del módulo.
Unitarias, integración, regresiones y cobertura	94 + 27 satisfactorias; 60,09 % existente / 90 % nueva.
Pipeline y reporte automático	Compilación, análisis, gate, carga/stress y artefactos CI verificados.
CI verde y fallido por problema real	Conservados; rojo como reproducción histórica controlada de DEF-001.
Smoke nivel 1 y scripts de carga/stress/esfuerzo	Ejecutables con comandos; perfiles y demo local satisfactorios.
Herramienta nueva con cinco pasos	k6 incorporado con PoC y matriz explícita.
Registro de defectos en gestor específico	GitHub Issues #7 a #10, vinculados con casos, corrección y evidencias; registro retrospectivo, cerrados por corrección previa.
Aporte individual trazable	Trabajo conjunto declarado; evidencia personal pendiente.
Integración externa y alcance académico	Contrato, dobles y base real local verificados; aceptación de localidad y alcance pendiente.
Despliegue nuevo y defensa	Nueva copia local demostrada; cloud no realizado y exposición del grupo pendiente.
Informe PDF	Este documento, con cifras derivadas de reportes y pendientes declarados.

Conclusión crítica

El módulo pasó de carecer de suite/pipeline medidos a contar con controles repetibles, cuatro defectos corregidos y evidencias verificables. La mejora funcional se apoya en tests estables, no solo en cobertura. El ajuste a cerca del 60 % fue una priorización solicitada; mantener un mínimo no elimina riesgos en caminos sin unitarias.

La deuda principal permanece en autenticación/autorización de lecturas, políticas operativas ante caída de base, validación de la app completa y ensayos más prolongados. El trabajo usa IA como asistencia; el grupo debe revisar y poder explicar scripts, resultados y límites.

Fuentes para la entrega

Índice y documentos versionados

Plan de pruebas y matrices

Registro completo de defectos

Política de cobertura y gate

CI roja/verde y pruebas idénticas

Resultados de carga

Los JSON originales, metadatos y parches están en docs/calidad/evidencias/. Los artefactos de Actions tienen retención de 30 días; las copias versionadas preservan esta entrega. No se incluyen credenciales ni datos reales de usuarios.